

Files / IO in Java

Presented by Sara Abbasghorbani



Advanced Programming - Spring 2026 - Dr Vahidi Asl





What will we learn in this session?



1) About Files

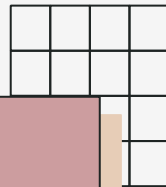
2) Stream

3) java.io

4) Write & Read

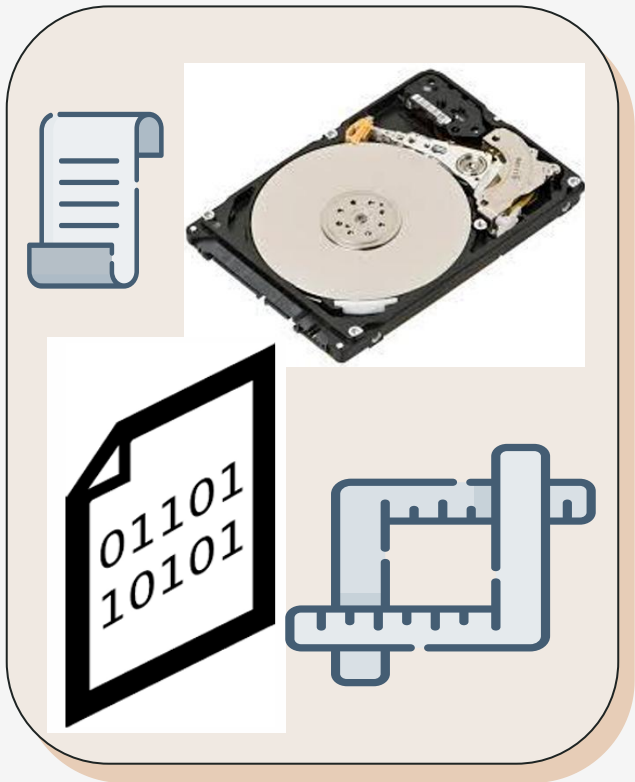
5) Input & Output Stream

6) JSON in Java



01

About Files



Types Of Files



This category is based on how to store it.

Text File:

The smallest part of it: Characters

Such as: txt, html, xml, ...



Binary Files:

Building blocks of it: Byte

Such as: exe, zip, pdf, docx

● ● ● Characters in Java



Java supports **Unicode standard** .

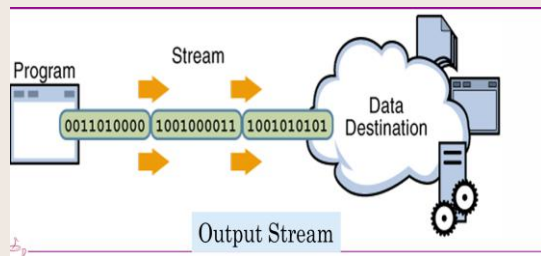
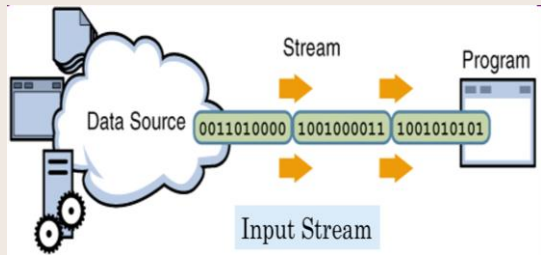
Unicode standard is the **set of characters** .

Each character is **stored as a number** in the computer.

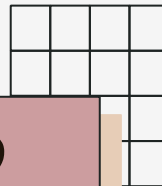
The character transformation method is determined by **coding techniques(Encoding)** .

Java uses **UTF-16 encoding** .

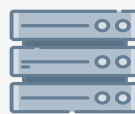
Each **UTF-16's character** is stored in **2 bytes** .



02



Stream



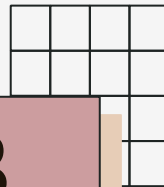
Stream

Flow of information into the program or out of a program:

- 1) Save and retrieve from file .**
- 2) Send information through the network .**
- 3) Exchange information with lateral systems(Scanner) .**

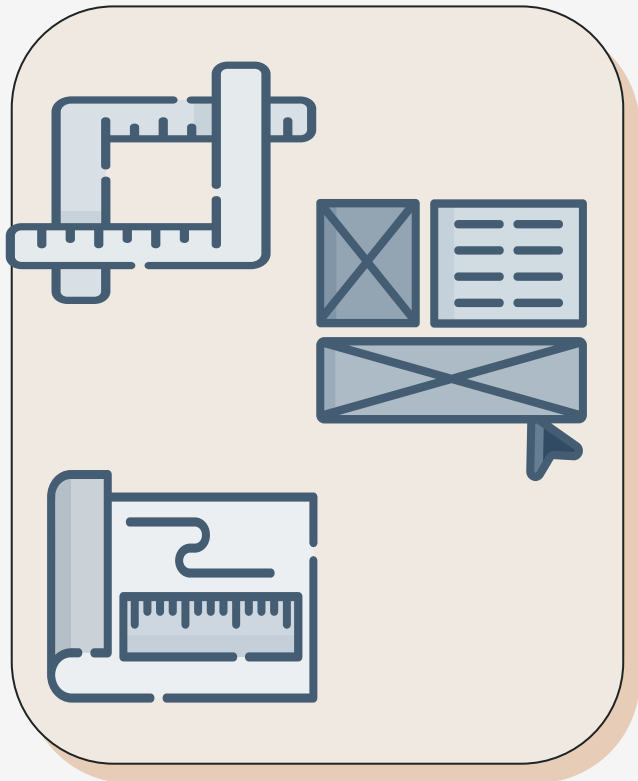
InputStream: What 's going on

OutputStream: What the program produces



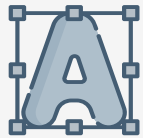
03

Java.io



The class of File in the java.io package lets us work with files. So we must build an object of this class and specify the file name or folder

```
import java.io.File; // Import the File class  
  
File f = new File("filename.txt"); // Specify the filename
```

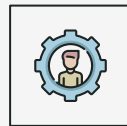


Some File class methods



mkdir() Boolean

Make directory



createNewFile() Boolean

Creat New File



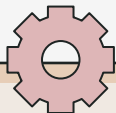
canRead() Boolean

Can the file be readable?



canWrite() Boolean

Can the file be written?



File's Methods

getName() String
Receive file name

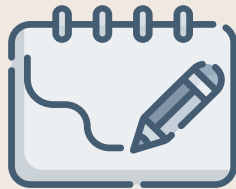
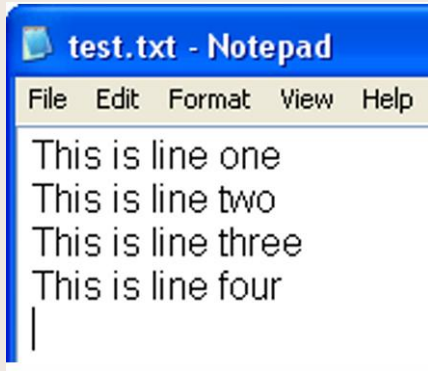
```
1 import java.io.File;
2 import java.io.IOException;
3
4 public class Methods_File {
5     public static void main(String[] args) throws IOException {
6         File folder = new File( pathname: "Downloads");
7         // create folder
8         System.out.println(folder.mkdir());
9         File file = new File( pathname: "Downloads");
10        // create file
11        System.out.println(file.createNewFile());
12        System.out.println("can read? " + file.canRead());
13        System.out.println("can write? " + file.canWrite());
14        System.out.println("exists? " + file.exists());
15        System.out.println("fileName: " + file.getName());
16    }
17 }
```

Methods_File x

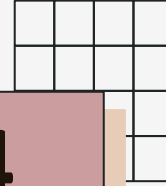


```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\J
true
false
can read? true
can write? true
exists? true
fileName: Downloads
```

exists() Boolean
Is there a file?



04



Write and Read from file

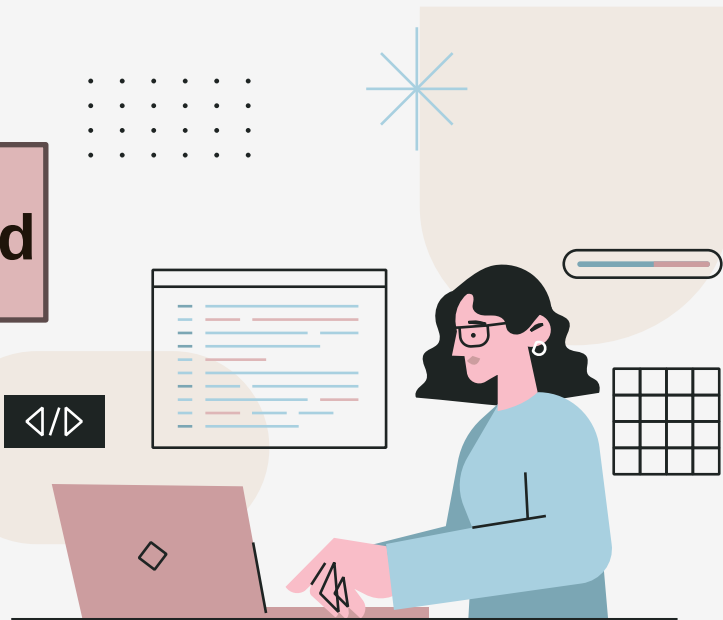


FileWriter class

write() method

Scanner class

Close() method



FileWriter

write()

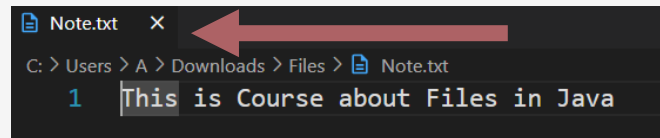
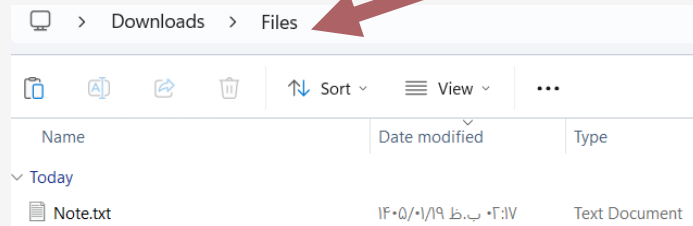
Close()



```
1 import java.io.File;
2 import java.io.FileWriter;
3 import java.io.IOException;
4
5 public class FileWriterExample {
6     public static void main(String[] args) throws IOException {
7         File folder = new File( pathname: "C:\\Users\\A\\Downloads\\Files" );
8         System.out.println(folder.mkdir());
9         File file = new File( pathname: "C:\\Users\\A\\Downloads\\Files\\Note.txt" );
10        System.out.println(file.createNewFile());
11        try {
12            FileWriter fileWriter = new FileWriter(file);
13            fileWriter.write( str: "This is Course about Files in Java");
14            fileWriter.close();
15            System.out.println("Successfully wrote to the file.");
16        } catch (IOException ioException){
17            System.out.println("An Error");
18            ioException.printStackTrace();
19        }
20    }
21 }
```

FileWriterExample x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\Intelli
true
true
Successfully wrote to the file.



Scanner



We can use Scanner to read from the file.

```
1 import java.io.File;
2 import java.io.FileWriter;
3 import java.io.IOException;
4 import java.util.Scanner;
5
6 public class ScannerExample {
7     public static void main(String[] args) throws IOException {
8         File file = new File( pathname: "Downloads\\Example.txt" );
9         FileWriter fileWriter = new FileWriter(file , append: true) ;
10        fileWriter.write( str: "\nIs it Example about FileWriter and Scanner");
11        fileWriter.close();
12        Scanner scanner = new Scanner(file) ;
13        while(scanner.hasNextLine()){
14            String line = scanner.nextLine();
15            System.out.println(line);
16        }
17        scanner.close();
18    }
19 }
```

The code indicates how to add text to the end of a file and then read and print the full file content. By setting true value for the second parameter FileWriter, the append state is activated to prevent the previous file content. Then, using Scanner, the line file is read to the line.

ScannerExample x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\I

Is it Example about FileWriter and Scanner

RandomAccessFile

RandomAccessFile class allows us to read or write in any situation, without having the previous content erased. Unlike FileWriter, which only writes from the beginning or end, this class has more flexibility to manipulate the file. To determine the position of writing or reading, the seek(long position) method is used.

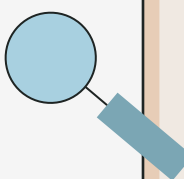


RandomAccessFile



```
import java.io.IOException;
import java.io.RandomAccessFile;

public class RandomAccessExample {
    public static void main(String[] args) throws IOException {
        RandomAccessFile randomAccessFile = new RandomAccessFile( name: "C:\\Users\\A\\Downloads\\Files\\Note.txt" , mode: "rw" );
        randomAccessFile.seek( pos: 5);
        randomAccessFile.writeBytes( s: "RandomAccessFile");
        randomAccessFile.close();
    }
}
```



Note.txt X

C: > Users > A > Downloads > Files > Note.txt

1 This is Course about Files in Java

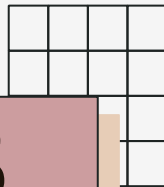
Note.txt X

C: > Users > A > Downloads > Files > Note.txt

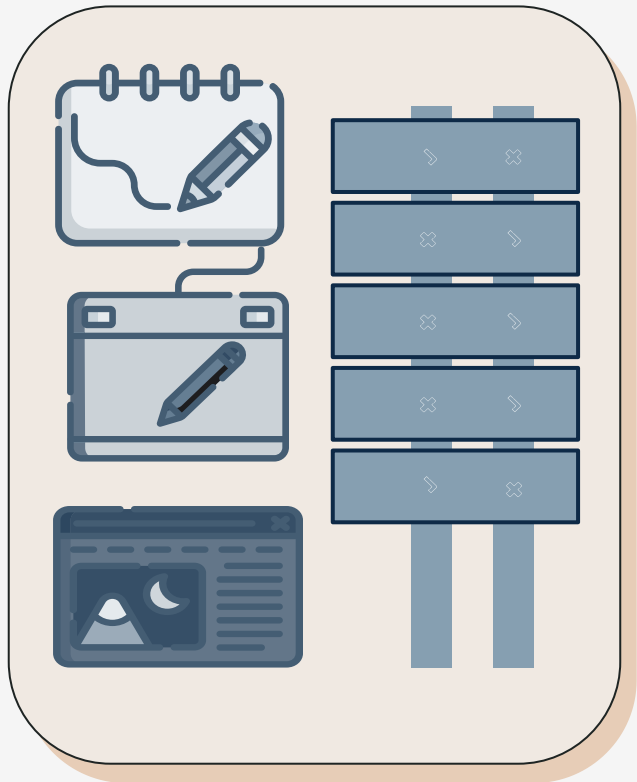
1 This RandomAccessFileFiles in Java



05



InputStream & OutputStream



Java Stream Classes

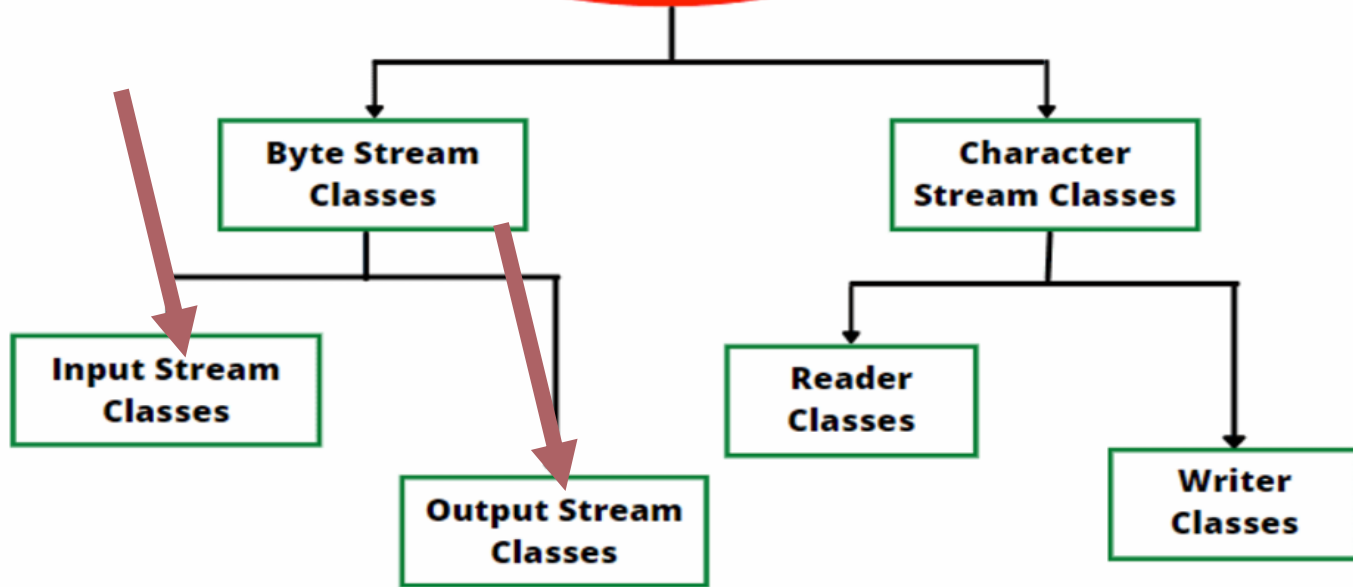
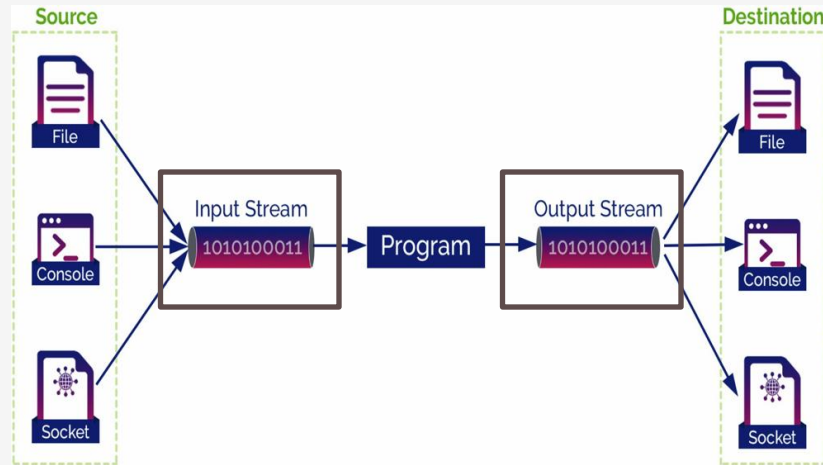


Fig: Classification of Java Stream Classes

What is Stream?



In computer science, **Stream** is a method for data transfer from one location to another. Usually, these data are **bytes**. Stream channels enable **reading and writing** data continuously, regardless of source or destination of data. The main advantage of the stream model is that the complexity of different data sources (**files, memory, network**) is hidden and provides a single method for data management allowing developers to work more efficiently with the i / o - bound operation.



FileInputStream



Read file by buffer array

```
1 import java.io.FileInputStream;
2 import java.io.IOException;
3
4 public class InputStreamExample {
5     public static void main(String[] args){
6         try(FileInputStream fileInputStream = new FileInputStream("name: \"Downloads\\Example.txt\")){
7             byte[] buffer = new byte[1024] ;
8             int bytes_read ;
9             System.out.println("Reading File Using Buffer");
10            while((bytes_read = fileInputStream.read(buffer)) != -1){
11                String text = new String(buffer , offset: 0 , bytes_read) ;
12                System.out.println(text);
13            }
14        } catch (IOException ioException){
15            System.out.println(ioException.getMessage());
16        }
17    }
18 }
```

File must be closed to release the system memory after opening. For this reason, using try-with-resources to close files automatically.

```
InputStreamExample x
Example.txt x
C: > Users > A > Downloads > Programming Languages > Java > Files and IO > Dow
1
2 Is it Example about FileWriter and Scanner
"C:\Program Files\Java\jdk-21\b
Reading File Using Buffer
Is it Example about FileWriter and Scanner
```



FileInputStream

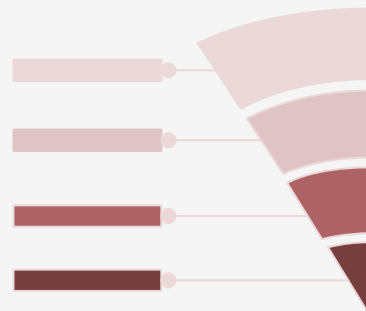
Read file by buffer array

```
Main.java Methods_File.java FileWriterExample.java ScannerExample.java RandomAccessExamp
1 import java.io.FileInputStream;
2 import java.io.IOException;
3
4 public class InputStreamExample {
5     public static void main(String[] args){
6         try(FileInputStream fileInputStream = new FileInputStream( name: "Downloads\\Input.txt")){
7             byte[] buffer = new byte[1024] ;
8             int bytes_read ;
9             System.out.println("Reading File Using Buffer");
10            while((bytes_read = fileInputStream.read(buffer)) != -1){
11                String text = new String(buffer , offset: 0 , bytes_read) ;
12                System.out.println(text);
13            }
14        } catch (IOException ioException){
15            System.out.println(ioException.getMessage());
16        }
17    }
18 }
```

InputStreamExample x

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Communit
Downloads\Input.txt (The system cannot find the file specified)"
```

Exception Handling



FileOutputStream



If the OutputStream.txt does not exist already, Java automatically makes it for us. However, if there is, the previous content is erased and replaced by the new content .

```
1 import java.io.FileOutputStream;
2 import java.io.IOException;
3
4 public class OutputStreamExample {
5     public static void main(String[] args){
6         String Text = "Hello, Students! Welcome to Java IO." ;
7         try(FileOutputStream fileOutputStream = new FileOutputStream( name: "Downloads\\OutputStream.txt")){
8             byte[] bytes = Text.getBytes() ;
9             fileOutputStream.write(bytes);
10            System.out.println("Data has been written to the file successfully!");
11        }catch(IOException ioException){
12            System.out.println("Error writing to file : " + ioException.getMessage());
13        }
14    }
15 }
```

OutputStream.txt X

C:\Users > A > Downloads > Programming Languages > Java > Files and IO > Downloads > OutputStream.txt

1 Hello, Students! Welcome to Java IO.

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition
Data has been written to the file successfully!



FileOutputStream



```
1 import java.io.FileOutputStream;
2 import java.io.IOException;
3
4 public class OutputStreamSecond {
5     public static void main(String[] args){
6         String New_Line = "\nThis line is appended to the existing file." ;
7         try(FileOutputStream fileOutputStream = new FileOutputStream("name: \"Downloads\\OutputStream.txt\" ", append: true)){
8             byte[] bytes = New_Line.getBytes() ;
9             fileOutputStream.write(bytes);
10            System.out.println("New data appended successfully!");
11        } catch(IOException ioException){
12            System.out.println("Error writing to file : " + ioException.getMessage());
13        }
14    }
15 }
```

OutputStreamSecond x

OutputStream.txt x

C: > Users > A > Downloads > Programming Languages > Java > Files and IO > Downloads > OutputStream.txt

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:
New data appended successfully!

```
1 Hello, Students! Welcome to Java IO.
2 This line is appended to the existing file.
```



FileOutputStream



```
1 import java.io.FileOutputStream;
2 import java.io.IOException;
3
4 public class ByteByteWriting {
5     public static void main(String[] args){
6         String text = "Java is fun !" ;
7         try(FileOutputStream fileOutputStream = new FileOutputStream("name: Downloads\\ByteByte.txt")){
8             System.out.println("Writing data byte by byte...");
9             for(int i = 0 ; i < text.length() ; i++){
10                 int character = text.charAt(i) ;
11                 fileOutputStream.write(character);
12             }
13             System.out.println("Done !");
14         } catch(IOException ioException){
15             System.out.println("Error is : " + ioException.getMessage());
16         }
17     }
18 }
```

We can write data in bytes. This method is not optimal for large files.

ByteByteWriting x

ByteByte.txt x

C:\> Users > A > Downloads > Programming Languages > Java > Files and IO > Downloads > ByteByte.txt

1 Java is fun !

"C:\Program Files\Java\jdk-21\bin\java.exe"

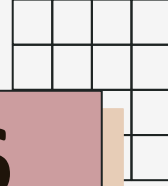
Writing data byte by byte...

Done !



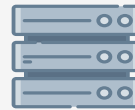


06



JavaScript Object Notation

JSON in Java



What is JSON?



J



S



The word JSON stands for JavaScript Object Notation. JSON is a very lightweight, text-based format for storing and transporting data. When we want to send data from a server to a client (like a web browser or mobile app), we usually use this format. Even though it has “JavaScript” in its name, JSON is completely language - independent and is supported in Java, Python, C#, etc.



O

N

• • • Main Rules of Writing JSON



1

Key-Value Pairs:

Data is always written as a key and a value. The key must always be enclosed in double quotes " ".

2

Comma Separation:

Different data elements are separated by commas , .

3

Curly Braces { } for Objects:

We use curly braces to define an object (which contains multiple properties).

4

Square Brackets [] for Arrays:

If we have a collection of data (a list), we put them inside square brackets. For example, an array with N elements is written as [item_0 , item_1 , ... , item_N]



```
1 {  
2   "student_id": 404243050,  
3   "first_name": "Mona",  
4   "last_name": "Maleki",  
5   "is_graduated": false,  
6   "courses": ["Advanced Programming", "Database", "Data Structure"],  
7   "grades": [18.5, 19.0, 20.0]  
8 }
```

In this example, "first_name" is a key and "Mona" is its value .
The "is_graduated" variable holds a boolean (logical) value .
The "courses" key is an array of strings (containing 3 courses) .
The "grades" key is an array of numbers.



Example



```
1 public class Student { no usages
2     String name ; 1usage
3     int age ; 1usage
4     public Student(String name , int age){
5         this.name = name ;
6         this.age = age ;
7     }
8 }
```

```
1 import com.google.gson.Gson;
2
3 public class JavaToJson {
4     public static void main(String[] args){
5         Student s1 = new Student( name: "Sara" , age: 20 ) ;
6         Gson gson = new Gson() ;
7         String JsonAnswer = gson.toJson(s1) ;
8         System.out.println(JsonAnswer);
9     }
10 }
```

JavaToJson x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Progr

{"name":"Sara","age":20}

```
1 import com.google.gson.Gson;
2
3 public class JsonToJava {
4     public static void main(String[] args){
5         String jsonString = "{\"name\":\"Sara\",\"age\":20}";
6         Gson gson = new Gson() ;
7         Student student = gson.fromJson(jsonString , Student.class) ;
8         System.out.println("Student name is : " + student.name);
9         System.out.println("Student age is : " + student.age);
10    }
11 }
```

JsonToJava x

"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\Je

Student name is : Sara

Student age is : 20

How to work with JSON in Java



To work with JSON in Java, we use the Gson library.

In this library, we deal with 2 very important operations:

1. **Serialization:** *Converting a Java Object into a JSON String.*
2. **Deserialization:** *The exact opposite; converting a JSON String back into a Java Object.*

```
1 import com.google.gson.Gson;
2
3 public class JavaToJson {
4     public static void main(String[] args){
5         Student s1 = new Student( name: "Sara" , age: 20 );
6         Gson gson = new Gson() ;
7         String JsonAnswer = gson.toJson(s1) ;
8         System.out.println(JsonAnswer);
9     }
10 }
```

Serialization (Java to JSON)

JavaToJson x

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\Java\jdk-21\bin\javaagent.jar" -classpath .;C:\Program Files\Java\jdk-21\bin\javaagent.jar JavaToJson
{"name":"Sara","age":20}
```

```
1 import com.google.gson.Gson;
2
3 public class JsonToJava {
4     public static void main(String[] args){
5         String jsonString = "{\"name\":\"Sara\",\"age\":20}";
6         Gson gson = new Gson() ;
7         Student student = gson.fromJson(jsonString , Student.class) ;
8         System.out.println("Student name is : " + student.name);
9         System.out.println("Student age is : " + student.age);
10    }
11 }
```

Deserialization (JSON to Java)

JsonToJava x

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\Java\jdk-21\bin\javaagent.jar" -classpath .;C:\Program Files\Java\jdk-21\bin\javaagent.jar JsonToJava
Student name is : Sara
Student age is : 20
```

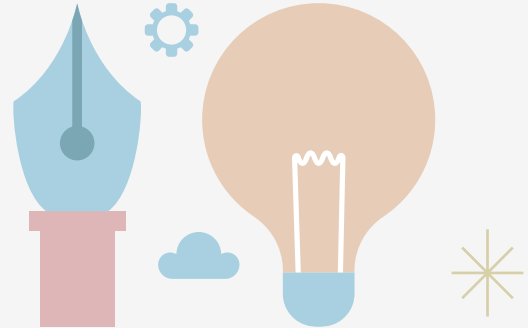
Resources

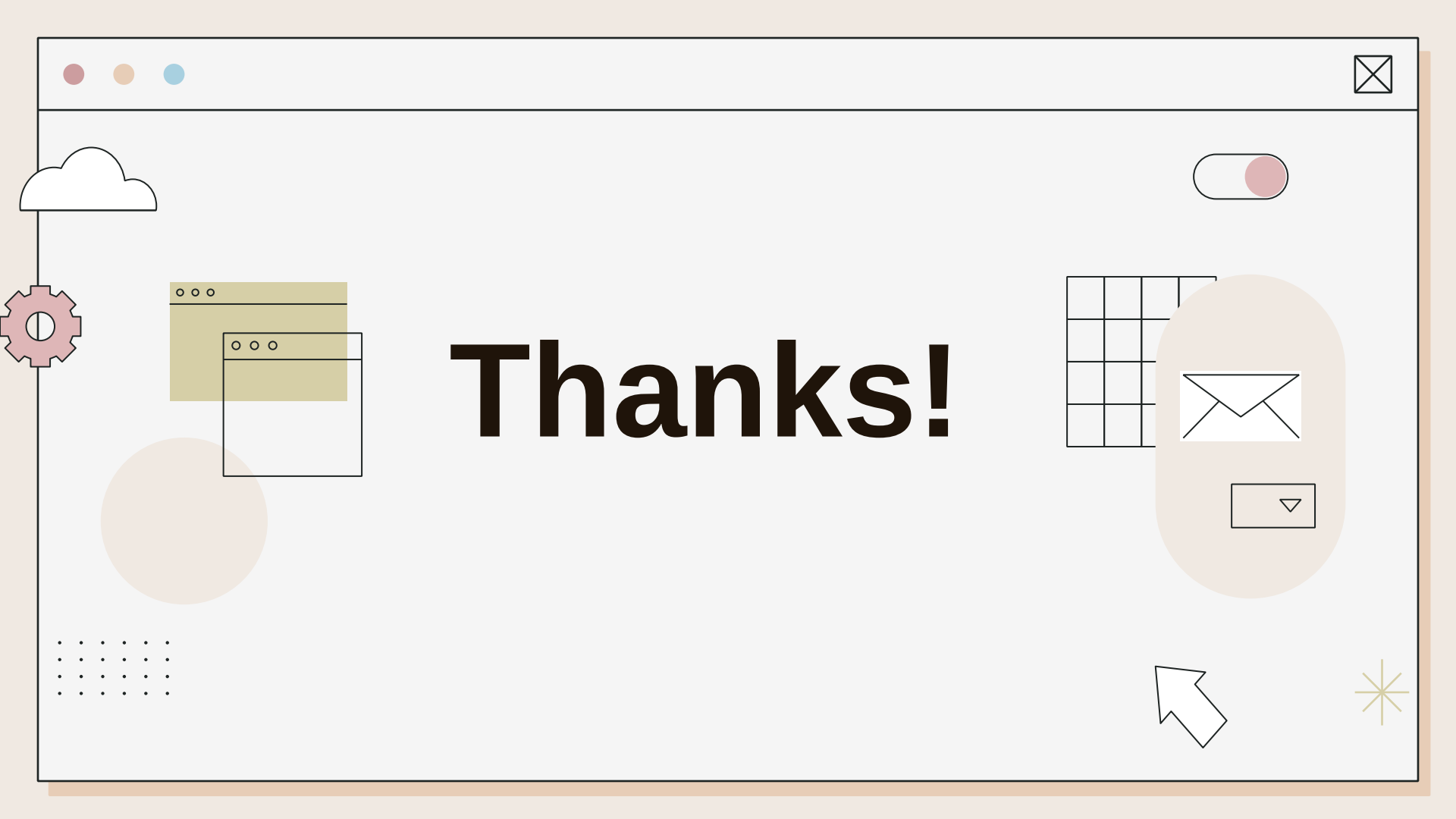


Project Oriented Course in Advanced Java Programming



IO



A stylized illustration of a computer window with a light gray border and a white background. The window has a title bar at the top with three colored circles (red, orange, blue) on the left and a close button (a square with an 'X') on the right. The main content area features the word "Thanks!" in a large, bold, black sans-serif font. Surrounding the text are various icons: a white cloud in the top left, a red gear on the left, two overlapping olive-green browser windows with three dots in their title bars, a large light-orange circle in the bottom left, a 4x4 grid of squares in the middle right, a red toggle switch in the top right, an envelope icon inside a light-orange rounded rectangle in the middle right, a small rectangle with a downward arrow below the envelope, a white mouse cursor arrow pointing up and to the left in the bottom right, and a yellow starburst in the bottom right corner. A 4x4 grid of small dots is in the bottom left corner.

Thanks!